From MDPs to *GRPO*

Four problems, four fixes: how reinforcement learning becomes a training signal for a language model, and how each limitation of one method motivates the next.

Every intermediate node is silent. The only lit node is the last one — that single fact drives most of what follows.

One idea leads to the next

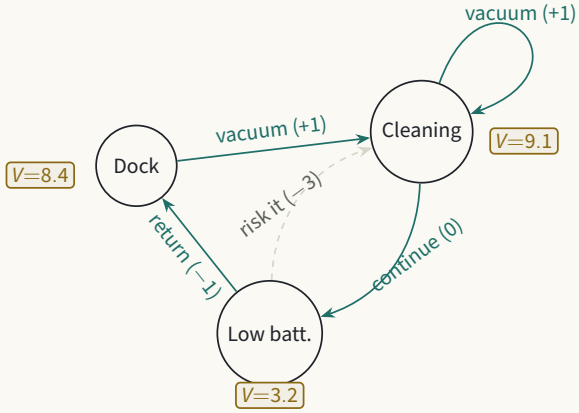
Problem	Reward only appears at the very end of a response.
Approach	**Policy gradients** — reinforce every token in a trajectory that scored well.
Problem	“Good” is meaningless without something to compare it to.
Approach	**Proximal Policy Optimization (PPO)** — score against a learned value baseline; clip large updates.
Problem	That value baseline is a whole second network to train.
Approach	**Group Relative Policy Optimization (GRPO)** — score completions against each other instead.
Problem	A reward model still needs human-labeled preference data.
Approach	**Reinforcement Learning with Verifiable Rewards (RLVR)** — let a deterministic checker score it.
Result	DeepSeek-R1-Zero reasons well but reads poorly — DeepSeek-R1 stages that into a usable model.

The vocabulary reinforcement learning starts with

A **Markov Decision Process (MDP)** is the tuple (S, A, P, R, γ). Everything else in this deck is a choice about how to fill in these six ideas.

SYMBOL	MEANING
$s \in S$	State — everything the agent currently knows.
$a \in A$	Action — a choice available from that state.
$P(s' \mid s, a)$	Transition — how the environment responds.
$r = R(s, a, s')$	Reward — the scalar signal returned by the environment.
$\pi(a \mid s)$	Policy — the agent's rule for choosing actions.
$\gamma \in [0, 1]$	Discount — how much future reward counts right now.

What that tuple actually looks like



Circles are states s, each carrying its value $V^{\pi}(s)$. Arrows are transitions under π, labeled with the action and reward. From *Low battery*, the policy prefers *return* over *risk it* because it leads toward the higher-value state.

The classical move: estimate value, then improve

$$V^{\pi}(s) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k} \mid s_t = s\right]$$
$$Q^{\pi}(s,a) = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^{k} r_{t+k} \mid s_t = s,\, a_t = a\right]$$

Classical RL usually assumes reward is available at, or near, every step — so value can be *bootstrapped*: propagated backward without waiting for an episode to end.

WHERE YOUR PRIOR PICTURE CAME FROM

Dynamic programming, TD-learning, ordinary actor–critic — all of them lean on dense, per-step feedback. That's exactly the assumption LLM training breaks.

Same tuple, new content

Token generation is still an ordinary episodic MDP. Only what fills each slot changes.

MDP SLOT	LLM INSTANTIATION
s_t	Prompt + tokens generated so far: $(x, y_1, \ldots, y_{t-1})$
a_t	The next sampled token, y_t
P	Deterministic append: $s_{t+1} = (s_t, a_t)$
π_θ	The model's own next-token distribution — its weights *are* the policy
T	Episode length — until an end token or a length cap

During rollouts we sample from this distribution rather than taking argmax. Sampling is what makes exploration possible.

PROBLEM

Reward only shows up at the very end

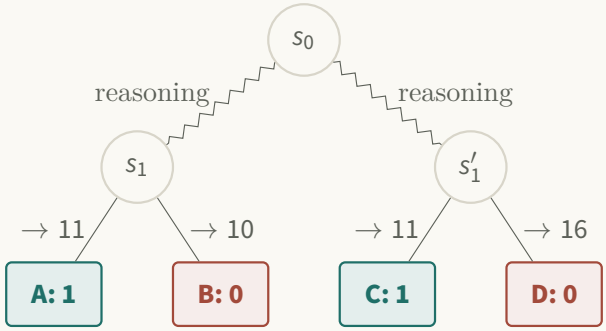
$$r_1 = r_2 = \cdots = r_{T-1} = 0, \quad r_T = R(\tau)$$

A verifier scores the *finished* response, not each token. Every token in a completion shares that one terminal reward, so credit assignment is crude: the model can't tell which specific token deserved it.

KEY DISTINCTION

This is the real gap from classical RL: not a different *kind* of problem, but one where the reward almost never shows up until the very end of the episode.

The same picture, redrawn for a language model



No cycles, no self-loops — each token choice is a branch, never a return, and no node carries a $V(s)$ the way *Dock* or *Cleaning* did. The wavy edges each compress *many* single-token states into one arrow; only the short straight edge into each leaf is one real token. Reward (green/red) exists only at the leaves.

BUILDING BLOCK

What are we even trying to maximize?

$$J(\theta) \quad = \quad \mathbb{E}_{x\sim D}\,\mathbb{E}_{y\sim\pi_\theta(\cdot|x)}\left[R(x,y)\right]$$

$J(\theta)$ is the average reward we'd get if we sampled forever from the current model, across the training prompts. Maximizing $J(\theta)$ means: make good responses more likely, bad ones less likely.

WHY THE LETTER J

There's no hidden meaning — J is just conventional RL notation for an objective function: "the quality of the policy parameterized by θ." Everything from here on is a piece we'll use to climb $J(\theta)$.

BUILDING BLOCK

Estimating its gradient from sampled rollouts

We can't compute $J(\theta)$ exactly — the space of possible completions is astronomically large. So instead of the true gradient, we estimate it using a handful of actual sampled rollouts. That's a **Monte Carlo** estimate: approximate an average using samples instead of the full computation.

$$\nabla_\theta J(\theta) \quad \approx \quad \frac{1}{N}\sum_{i=1}^{N} R(\tau_i) \sum_t \nabla_\theta \log \pi_\theta(a_{i,t} \quad | \quad s_{i,t})$$

WHY THIS IS ALLOWED

This estimate is unbiased for *any* N, including $N{=}1$ — that's what licenses training on a small sampled batch instead of the intractable full sum.

BUILDING BLOCK

Zooming into a single sampled token

Drop the sums over samples and time, and look at what happens to just one token. This is the piece both PPO and GRPO are built on:

$$\theta \quad \leftarrow \quad \theta + \alpha \underbrace{\nabla_\theta \log \pi_\theta(a_t \mid s_t)}_{\text{token-level piece}} R$$

α is the learning rate; the gradient term points toward higher probability for the token actually sampled; R scales how hard to push, and in which direction. Sum this over every token in a response for the full update.

PROBLEM

Reward alone can't tell you if an action was good

Two prompts: an easy one where most completions score 1, a hard one where most score 0. A completion scoring 1 on the easy prompt may be unremarkable; a completion scoring 0 on the hard prompt may be one of the better attempts made. Raw $R(\tau)$ can't express "good, relative to what this prompt usually produces."

$$A_k \quad = \quad R_k \; - \; \text{baseline}(s_k)$$

THE QUESTION EVERY APPROACH BELOW ANSWERS DIFFERENTLY

PPO and GRPO are, at heart, two different answers to one question: what should baseline(s) actually be? Once we know, R in block 3 gets replaced by this advantage A.

BUILDING BLOCK

Keeping the model close to a reference

π_θ isn't a distribution — it's a function from states to distributions. But fix one specific state s_t, and both $\pi_\theta(\cdot|s_t)$ and $\pi_{\text{ref}}(\cdot|s_t)$ become two ordinary, comparable distributions over the same vocabulary. **That** is what KL divergence compares.

$$\mathrm{KL}_t \quad = \quad \mathrm{KL}\big(\pi_\theta(\cdot \mid s_t) \,\|\, \pi_{\text{ref}}(\cdot \mid s_t)\big)$$

COMPUTED HOW, IN PRACTICE

Per token, reusing only the sampled token's probability under each model — the same two numbers used for the ratio below — via a low-variance single-sample estimator, not a full sum over the vocabulary.

Why PPO needs a rewrite, not just the raw gradient

Rollouts are expensive, so PPO takes *several* gradient steps on the same batch before sampling again. But block 3's gradient is only valid at the exact θ that generated the data — the moment θ moves, it stops being correct for this data. The fix is **importance sampling**: reweight old samples to estimate an expectation under the new policy.

$$\mathbb{E}_{a\sim\pi_\theta}\big[A(s,a)\big] \quad = \quad \mathbb{E}_{a\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(a\mid s)}{\pi_{\text{old}}(a\mid s)}\cdot A(s,a)\right]$$

WHY REUSE IS EVEN LEGITIMATE — NOT IN THE BOOK

A sampled token isn't owned by the policy that produced it — any policy assigning it nonzero probability could have generated it. So after θ moves, the same recorded token is still valid data; we just ask how likely the *current* policy thinks it was, and reweight by the ratio. §7.3 presents the ratio as a stabilizer without explaining why reusing old samples this way is sound.

The ratio is block 3's gradient, in disguise

$$\nabla_\theta \left[\text{ratio}_t(\theta) \cdot A_t\right] = \nabla_\theta \left[\frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)}\right] \cdot A_t \qquad \textit{differentiate the ratio; } \pi_{old} \textit{ is fixed}$$

$$= \frac{\pi_\theta(a_t|s_t)}{\pi_{\text{old}}(a_t|s_t)} \cdot \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A_t \qquad \textit{identity } \nabla f = f \cdot \nabla \log f\textit{, again}$$

$$\text{at } \theta = \theta_{\text{old}}\text{:} \quad \text{ratio}_t = 1 \implies \nabla_\theta L(\theta)\big|_{\theta_{\text{old}}} = \nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A_t$$

WHY BOTHER, IF THEY AGREE AT THE START

Because $L(\theta)$ stays valid *after* θ has moved a few steps — you can keep differentiating it for several updates on the same batch. And $\text{ratio}_t(\theta)$ has a readable meaning ("how much more or less likely is this action now?"), so it can be directly clipped. That clip is the *P* in Proximal.

PROBLEM

The rewrite only holds up close to θ_{old}

The last slide showed $\nabla_\theta L(\theta)$ gives back the *exact* right gradient — but only exactly at $\theta = \theta_{\text{old}}$, where $\text{ratio}_t = 1$. As the optimizer takes those several steps from bridge A, θ moves away, and $\text{ratio}_t(\theta)$ can drift far from 1 for some tokens. Nothing in $L(\theta) = \text{ratio}_t(\theta) \cdot A_t$ stops it from being pushed further and further in a direction that looks good on paper, using a policy comparison that's no longer trustworthy.

> **NAMING THIS: THE TRUST REGION**
>
> The zone around θ_{old} where $\text{ratio}_t(\theta)$ stays close to 1 — where the rewrite is still a reasonable stand-in for the real gradient — is called the **trust region**. PPO needs some way to stop an update from pushing $\text{ratio}_t(\theta)$ far outside it. That's what clipping is for.

BUILDING BLOCK

What the clip actually does

Two versions of the same term get computed — the true one, and one where the ratio is capped to the trust region. min always keeps whichever is *smaller*:

$$L_t \quad = \quad \min\Big(\underbrace{\text{ratio}_t(\theta)\,A_t}_{\text{true value}},\ \underbrace{\text{clip}\big(\text{ratio}_t(\theta),\,1-\varepsilon,\,1+\varepsilon\big)\,A_t}_{\text{value if ratio is capped to the trust region}}\Big)$$

Scenario ($A=+1$, band $[0.8, 1.2]$)	**ratio**	**true**	**capped**	**min picks**
Pushed further, past the margin	1.5	1.5	1.2	capped — flat
Drifted the wrong way	0.7	0.7	0.8	true — uncapped

THE ONE-LINE VERSION

Clipping caps the reward for overshooting a good move. It never caps the penalty for a bad one.

Is this still mathematically correct?

No — and that's intentional. Once the ratio is capped, clip$(\cdot)$ is a *constant* with respect to θ, so its gradient is exactly zero there: L^{CLIP} discards the true importance-sampling gradient rather than approximating it. It is not an unbiased estimator of $\nabla_\theta J(\theta)$.

WHY THIS IS A DELIBERATE TRADE, NOT SLOPPINESS

- Raw importance sampling is already unreliable once the ratio drifts far — a handful of samples end up dominating the estimate.
- Grounded in real theory: optimizing a *pessimistic* (lower-bound) surrogate while limiting how far θ moves guarantees $J(\theta)$ doesn't get worse (Kakade & Langford; TRPO). PPO's clip is a cheap stand-in for TRPO's exact, expensive version.
- Nothing upstream was exact past θ_{old} anyway — clipping is one more deliberate approximation, chosen for stability.

PPO was never claiming exactness beyond θ_{old} — clipping just makes the approximation fail *safe* instead of failing silently.

PPO's baseline: "better than expected," not "good"

$$A_t \quad = \quad G_t \; - \; V_\phi(s_t)$$

Quantity	Value
$V_\phi(s_t)$	0.9 — this prefix usually leads to strong outcomes
Sampled G_t	0.6 — a positive, reasonable-looking result on its own
A_t	$0.6 - 0.9 = -0.3 \Rightarrow$ this token gets *suppressed*

WHY THIS MATTERS

A completion with a decent, positive reward can still get pushed down. The value model isn't asking "was this good?" — it's asking "was this better than what this prefix already tends to produce?" This $V_\phi(s_t)$ is PPO's specific choice of baseline(s) from the noisy-reward problem.

APPROACH

Proximal Policy Optimization (PPO) — the complete update rule

$$L_{\text{PPO}}(\theta) = \mathbb{E}_t\Big[\min\big(\text{ratio}_t(\theta)\,\tilde{A}_t,\ \ \text{clip}(\text{ratio}_t(\theta),\,1-\varepsilon,\,1+\varepsilon)\,\tilde{A}_t\big)\Big]$$

$$\text{where}\quad \tilde{A}_t = \tilde{G}_t - V_\phi(s_t), \qquad \tilde{r}_t = r_t - \beta\,\text{KL}_t(\pi_\theta \| \pi_{\text{ref}})$$

Every piece is already familiar: ratio_t and clip are block 5; $V_\phi(s_t)$ is the value-model baseline from the last slide; the KL penalty is block 4, folded into the reward before $\tilde{A}_t$ is ever formed.

PROBLEM

The value model is a whole second network

V_ϕ needs its own forward pass, its own regression loss, its own optimizer state. For a frontier-scale LLM, that isn't a footnote — it's a large fraction of training cost.

> **THE QUESTION GRPO ASKS**
>
> Can we get a usable, prompt-specific baseline without *learning* one?

APPROACH

Group Relative Policy Optimization (GRPO)

GRPO answers the same baseline question as PPO, but without training a value model. Instead: sample several completions for the *same* prompt, and compare each one to the average of the group.

> **THE TREE FROM A FEW SLIDES AGO**
>
> That's exactly the branching tree we drew for token generation — four completions (A, B, C, D) from one prompt. GRPO scores all of them and compares, instead of training a separate network to predict how well any one of them "should" do.

Sample a group, compare within it

$$A_i = \frac{r_i - \text{mean}(r_1, \ldots, r_G)}{\text{std}(r_1, \ldots, r_G) + \varepsilon}$$

Completion	Final answer	Reward
A	11	1
B	10	0
C	11	1
D	16	0

Group mean $= 0.5$. The verifier only ever said 0 or 1 — it's the comparison against the group that turns that into a signal with both sign and magnitude: A, C reinforced; B, D suppressed.

GRPO — the complete update rule

$$L_{\text{GRPO}}(\theta) \;=\; \mathbb{E}_i\Big[\min\big(\text{ratio}_i(\theta)A_i,\;\; \text{clip}(\text{ratio}_i(\theta),\,1-\varepsilon,\,1+\varepsilon)A_i\big)\Big] \;-\; \beta\cdot \mathrm{KL}(\pi_\theta \| \pi_{\text{ref}})$$

$$\text{where}\quad A_i \;=\; \frac{r_i - \text{mean}(r_1,\ldots,r_G)}{\text{std}(r_1,\ldots,r_G)+\varepsilon}$$

ratio_i and clip are the *exact same* block 5, unchanged. A_i is the group-relative advantage from the last slide, replacing PPO's value model. KL is added as its own term here, instead of folded into the reward.

THE TRADE-OFF

GRPO isn't cheaper because it samples fewer completions — it samples *more*. At matched sample count, it's cheaper because there's no critic path, and KL as its own term keeps its effect explicit and easier to tune.

APPROACH

Reinforcement Learning with Verifiable Rewards (RLVR)

Even under GRPO, a reward model still has to be trained on human-labeled preference data. For math, code, and other objectively checkable tasks, RLVR skips that: $R(x, y)$ comes from a deterministic verifier — compare a final answer, run unit tests, check a formal equivalence.

VERIFIABLE

Math, code, symbolic tasks

Deterministic checker: correct/incorrect, unit tests pass/fail.

NOT VERIFIABLE

Helpfulness, safety, style

Not mechanically checkable — still needs learned reward models or preference data.

One base model, two independent paths

DeepSeek-V3-Base is the pretrained checkpoint before any reasoning-specific training — it predicts text well, but hasn't yet been pushed toward long, self-corrective reasoning. Two separate recipes both start from this *exact same* checkpoint, built independently of each other:

NEXT SLIDE

Pure RL → R1-Zero

GRPO + RLVR, straight from the base.

A FEW SLIDES AHEAD

Staged pipeline → R1

Cold-start data, then RL, then filter, then RL again.

THE POINT TO HOLD ONTO

Neither path continues the other's weights. R1-Zero and R1 are trained independently — even though, as you'll see, they end up sharing some data.

RESULT

GRPO + RLVR, applied directly to the base model

DeepSeek-R1-Zero starts from DeepSeek-V3-Base and applies GRPO with RLVR directly — no supervised chain-of-thought examples first. Rewarded only for solving verifiable problems, the model begins allocating more tokens to hard problems, checking its own intermediate steps, and revising when something looks wrong — often called an *“aha moment.”*

> **BUT**
>
> The report also notes two problems: poor readability, and language mixing within a single response. Why those specific problems, on the next slide.

PROBLEM

Why “correct” isn’t the same as “readable”

R1-Zero’s reward checks exactly one thing: is the final answer correct (plus a loose check that reasoning sits inside the <think> tags). Nothing in that signal cares how the reasoning *in between* reads.

POOR READABILITY

No reward for clean, organized prose — only for reaching the right number. Reasoning can be repetitive and still earn full credit.

LANGUAGE MIXING

The base model is multilingual; nothing penalizes switching languages mid-response, so English can drift into Chinese and back.

WHAT A FIX REQUIRES

Not running R1-Zero’s recipe for longer — changing *what gets rewarded* and *how the model starts*.

BUILDING BLOCK

What is Supervised Fine-Tuning (SFT)?

Everything else in this deck has been reinforcement learning: sample, score, reinforce. SFT is different — it's *imitation* learning. Collect a dataset of (prompt, correct response) pairs, written by humans or filtered from a model's own outputs, and train with ordinary next-token prediction to reproduce those exact responses.

	SFT	RL (GRPO / PPO)
Data	(prompt, correct answer) pairs	sampled completions + a reward
Objective	match the given answer, token by token	maximize expected reward
Exploration	none — pure imitation	model must sample and discover

WHY R1 USES IT

To give the model a good starting *style* before RL ever begins, and to blend reasoning back in with everyday assistant skills partway through the pipeline.

APPROACH

DeepSeek-R1: built fresh, informed by R1-Zero's outputs

R1 does *not* continue R1-Zero's weights. It starts from the same DeepSeek-V3-Base, and runs its own staged pipeline: cold-start SFT → reasoning RL, now with a language-consistency reward → rejection sampling + mixed SFT with general data → a final hybrid-reward RL stage.

NOT THE SAME WEIGHTS

The only thing R1-Zero contributes is *data*: some of its own outputs, sampled and heavily filtered for quality, become part of R1's cold-start training set. R1-Zero's parameters are never copied.

Which stage fixes which problem

Stage	Adds	Fixes
Cold-start SFT	A small set of curated, readable reasoning examples	No readable starting style
Reasoning RL	Same GRPO+RLVR as R1-Zero, plus a reward for staying in one language	Language mixing
Rejection sampling + mixed SFT	Filter for correct *and* readable; mix with general instruction data	Over-narrow specialization
Final hybrid RL	Rule-based rewards where verifiable, learned preference rewards elsewhere	Not aligned as a general assistant

Each stage is a targeted patch on a specific gap left by the one before it — the pipeline isn't one big fix, it's four small ones.

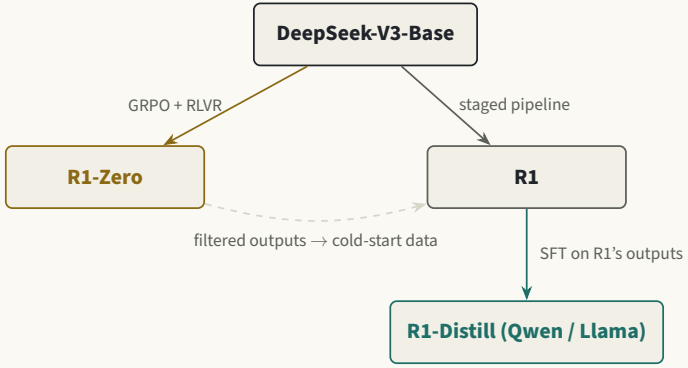
APPROACH

DeepSeek-R1-Distill: smaller models, no RL at all

R1-Distill isn't a DeepSeek architecture continued — it's existing smaller models from *other* labs (Qwen from Alibaba, Llama from Meta), taken as pretrained checkpoints and given ordinary supervised fine-tuning on a large set of reasoning traces generated by the finished R1 model. No RL, no verifier, no reward — pure imitation of R1's outputs.

> **WHY**
>
> Deployment economics. R1 itself is large and expensive to run; a 7B model fine-tuned to imitate its reasoning style is far cheaper, even if somewhat less capable.

The whole family, in one picture



Solid arrows are real training — new weights come out the other end. The dashed arrow is data only: R1-Zero's filtered outputs feed R1's training set, but R1's weights never come from R1-Zero's checkpoint.

One scalar loss, one backward pass

$$L(\theta) \quad = \quad -A \sum_{t=1}^{T} \log \pi_\theta(a_t|s_t)$$

```
# log_probs holds one term per generated token
loss = -advantage * log_probs.sum()
loss.backward()
optimizer.step()
```

backward() does not need to be called per token. Differentiation is linear, so autograd accumulates every token's gradient contribution into one computation graph automatically.

> **THE SHORT VERSION**
>
> Same principle as minibatch SGD: sum the per-example losses into one scalar, then call backward() once.

The whole lineage, one line each

REINFORCE	Reinforce every sampled token by how well the whole response scored.
+ baseline	Compare against what's typically expected, not the raw score.
PPO	The baseline is a learned value model; KL is baked into the reward; updates are clipped.
GRPO	The baseline is the average of a sampled group; KL is its own loss term; no critic to train.
GRPO + RLVR	For verifiable tasks, even the reward itself comes from a deterministic checker.

THE ONE IDEA UNDERNEATH ALL OF IT

Every step nudges sampled tokens up or down by how good they were relative to expectation. What changes, each time, is only where "expectation" comes from.